

VERIHARNESS: Scaling Agentic Verification for Long-Horizon Tasks

Caiqi Zhang^{1*}, Rujun Han², Zifeng Wang², Zoey CuiZhu², Nigel Collier¹, Tomas Pfister² and Chen-Yu Lee²

¹University of Cambridge, ²Google Cloud AI Research

As LLM agents undertake increasingly complex, long-horizon tasks, verifying their outputs becomes increasingly challenging. We study how verification capability can be strengthened with a fixed base model, without access to reference answers or grading rubrics at test time. Repeated sampling yields multiple rollouts that can contain complementary correct claims, but we need a reliable verification mechanism to determine which claims to trust. We first find that disagreement often exposes correct alternatives, while consensus can conceal errors. These observations motivate VERIHARNESS, which turns the underlying LLM a generator uses into an agentic verifier by giving it a workspace, evidence tools, and reusable verification skills. A disagreement resolver checks competing claims against environmental evidence, while a consensus challenger tests shared claims and searches for omitted requirements. Their findings guide the selection and revision of the final artifact. Across five long-horizon workspace benchmarks and two frontier models, VERIHARNESS achieves the highest selection scores among the evaluated baselines. Evidence-backed revision further improves average performance, bringing gains over a single rollout to 6.2 points with Gemini 3.5 Flash and 6.4 points with Claude Opus 4.8. We further show that verification skills can self-improve from failure feedback, demonstrating VERIHARNESS as a novel and critical approach for scaling long-horizon agentic verification. We release the full pool of approximately 26,000 rollouts across all five benchmarks and both models, produced at a cost of over \$100,000, to support future research on agentic verification.

[Code](#) | [Dataset](#) | veriharness.com

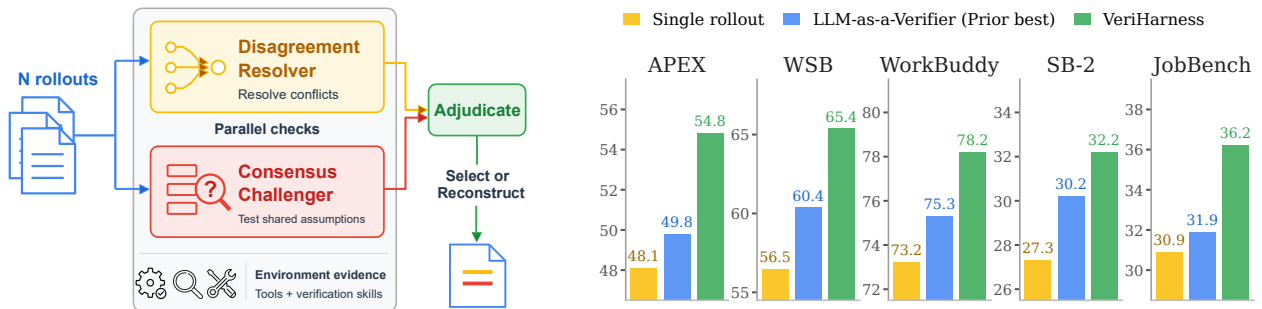


Figure 1 | **VERIHARNESS and its results.** *Left:* the verifier is the generator’s own model inside a harness. A disagreement resolver tests the claims on which the N rollouts differ, and a consensus challenger seeks evidence against the claims they share. Adjudication combines their findings to guide artifact selection and revision. *Right:* with Gemini 3.5 Flash as both generator and verifier, VERIHARNESS with revision improves over the single-rollout baseline on all five benchmarks, including gains of **6.7 points** on APEX-Agents and **8.9 points** on Workspace-Bench Lite (WSB).

1. Introduction

Long-horizon agents increasingly produce reports, spreadsheets, and other artifacts whose correctness requires substantial work to assess (Tang et al., 2026; Vidgen et al., 2026; WorkBuddy Team et al.,

* This work was done while Caiqi interned at Google Cloud AI Research.

Corresponding author(s): cz391@cam.ac.uk, {rujunh, chenyluee}@google.com

2026). Recent workspace benchmarks show that even frontier models struggle to produce reliable artifacts (Li et al., 2026b; Zhu et al., 2026). Verifying these artifacts is harder still. In coding and mathematics, unit tests, executable checks, or formal proof checkers give direct feedback on correctness even when no reference answer exists (Ehrlich et al., 2025; Li et al., 2025). General complex tasks rarely offer an equally straightforward way to assess the entire output. For example, checking a report may require tracing claims to sources, recomputing quantities, and interpreting requirements across several files. As agents take on more of this work, verification becomes a central challenge and must develop alongside generation (You et al., 2026).

We study test-time verification without a stronger judge: the generator and verifier share the same model and tools, and the verifier checks the output artifacts without access to reference answers or grading rubrics. This setting reflects practice at the frontier, where the generator is already the strongest available model and no stronger judge exists to oversee it (Bowman et al., 2022; Burns et al., 2024). Holding the base model fixed also ensures that no knowledge from a stronger model enters the verifier: any gain comes from the model’s own rollouts and environment (Huang et al., 2023). In this setting, the verifier draws on two unique sources of information: 1) the candidate rollouts and 2) the task environment. Repeated sampling yields several rollouts of the same task (Brown et al., 2024; Zhu et al., 2025), and comparing them reveals competing claims and shared conclusions. The task environment provides source files, data, and constraints against which those claims can be checked.

Existing test-time verification methods mostly use only the rollouts. Majority voting and self-consistency (Chen et al., 2023; Wang et al., 2023) treat the most common answer as correct, and LLM-as-a-judge approaches (Kwok et al., 2026; Zheng et al., 2023) score rollouts by reading them. Both rely on what the model already believes rather than on evidence the environment could provide. To address this shortcoming, we first propose to bring verification into the environment: individual claims are tested against the workspace’s source data and constraints. Second, we find that **consensus does not guarantee correctness, while disagreement between rollouts can expose correct alternatives**. A case study in Figure 2 shows that roughly a third of the agreed values are judged incorrect, so errors cannot be corrected by voting or selection alone. Also, disputed claims often include a correct candidate, although the most frequent answer may be wrong (Section 2). Checking these alternatives against evidence can help identify the correct value. Therefore, a verifier has to test competing answers and challenge the consensus that all candidates share.

We introduce VERIHARNES (Figure 1), a general-purpose, plug-and-play harness for the verifier, which uses the same underlying model as the generator. The harness encompasses a workspace, evidence tools, and reusable verification skills to support these two checking tasks. A *disagreement resolver* tests competing claims against source files, data, and task constraints. A *consensus challenger* searches for evidence that could refute consensus values or reveal omitted requirements. Their findings guide the selection and revision of the final artifact, accompanied by a record of the supporting evidence. The verifier decides what to check, how to interpret the evidence, and which changes to make. We further ask whether the harness can grow its verification skills with experience. Building on reusable skill and experience libraries (Ouyang et al., 2026; Wang et al., 2024), we let the harness accumulate skills from failure feedback on development tasks, starting from either an

Consensus claims ($H = 0$, $n = 917$)

Agreed answer



Disputed claims ($H > 0$, $n = 775$)

Correct candidate available



Most frequent answer correct



Figure 2 | **Consensus can preserve errors; disagreement can contain correct alternatives that voting discards.** APEX-Agents claims from ten-rollout pools of Claude Opus 4.8; percentages are within each group, correctness uses the benchmark’s rubric grades, and H is the claim entropy of Section 2.

empty library or a human-authored one.

We evaluate on five demanding workspace benchmarks with two frontier models, Gemini 3.5 Flash and Claude Opus 4.8, each serving as both generator and verifier. VERIHARNES achieves the highest selection score among the evaluated baselines on every benchmark with both models. Evidence-backed revision further improves average performance, yielding gains over a single rollout of 6.2 points with Flash and 6.4 with Opus (Table 2). On held-out tasks, skills evolved from an empty library outperform the human-authored library, and evolving from the human-authored library adds 6.8 points on APEX-Agents and 3.7 on SpreadsheetBench 2 over that library (Section 5).

Our contributions are threefold:

- **Verification principle.** We identify two complementary checking tasks in same-model verification: resolving disputed claims and challenging consensus claims against evidence from the environment.
- **General-purpose harness.** We build VERIHARNES, the first agentic verification harness for long-horizon tasks, which is training-free and plug-and-play across benchmarks and models, and evaluate its ability to select and revise agent outputs.
- **Evolving verification skills.** We show that a fixed model can accumulate useful verification skills from failure feedback, with human expertise providing an effective starting point for evolution.

2. Problem Formulation

Task and rollouts. A task $t = (s, \mathcal{E})$ consists of a description s and an environment $\mathcal{E}$ containing source files, data, and application state. The *generator* is an agent built on model M that attempts the task. Each attempt produces a *rollout* $r_i = (d_i, w_i, \tau_i)$, comprising a delivered artifact d_i , final workspace state w_i , and action trace τ_i , and repeated sampling yields a pool $R = \{r_1, \dots, r_N\}$. An artifact makes *claims*: assertions about task-relevant values, interpretations, or the satisfaction of requirements, such as a reported figure, the reading of a clause, or a required section.

Verification objective. The *verifier* uses the same model and tools as the generator. Given the task and the pool, it delivers a rollout r^* , selected from R or revised from one of its members, together with a record C of checks, findings, and unresolved issues. Its objective is to maximize the quality of r^* as judged by an external grader. The verifier has no access to this grader, to reference answers, or to grading rubrics, and all compared methods receive the same rollout pool.

Checks and evidence. We examine artifacts at the level of claims. A *check* χ is an operation that tests a claim, such as reading a source passage, recomputing a total, or inspecting file metadata, and its result $e = \mathcal{E}(\chi)$ is the evidence obtained by running the check in the environment. For example, “FY2025 revenue is EUR120m” is a claim, and inspecting the final report for the amount and currency is a check. Evidence acquisition is *active*: the verifier chooses each check from the task, the candidate pool, and the evidence gathered so far. An informative check helps distinguish candidate answers or test a constraint that the current answer may violate. Evidence can support a candidate, contradict it, or establish a correction absent from the pool, and a claim remains unresolved when the evidence is insufficient for a verdict. Task requirements also identify claims to examine when the corresponding content is absent from every artifact.

Consensus and disputed claims. Let p identify a common question or requirement, and let $v_{i,p}$ denote the answer or content supplied by artifact d_i , treating equivalent expressions as the same value and omission as a distinct value. The candidate set $\mathcal{V}_p = \{v_{i,p}\}_{i=1}^N$ has empirical distribution and

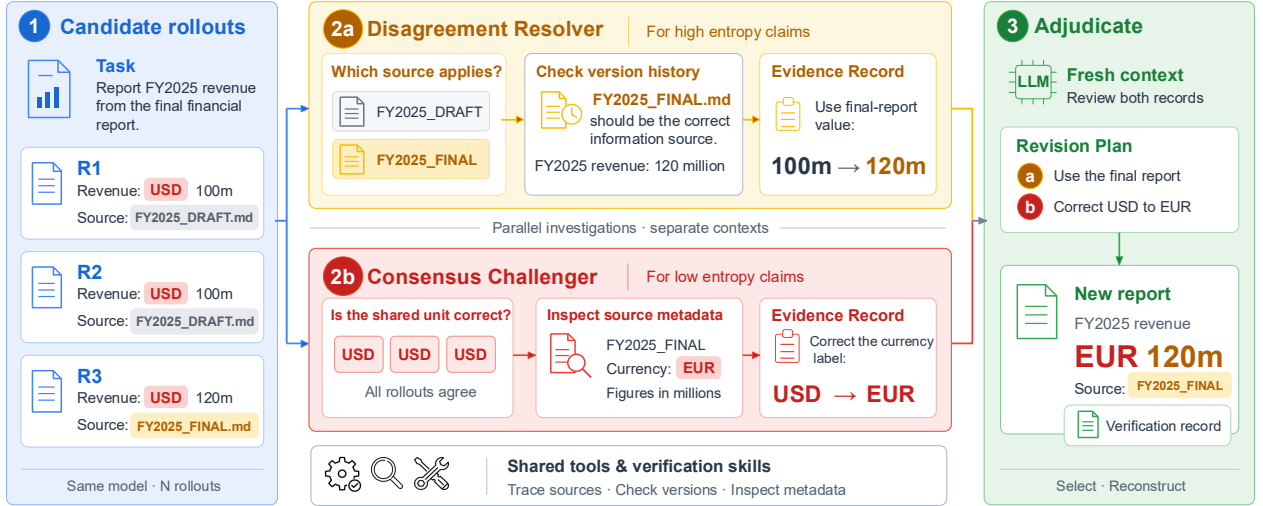


Figure 3 | **VERIHARNESS on a worked example.** The task asks for FY2025 revenue from the final financial report. (1) Two rollouts cite the draft and report 100m; a third cites the final report and gives 120m, and all three label the figure USD. (2a) The resolver checks version history and establishes that the final report supersedes the draft, supporting 120m. (2b) In a separate context, the challenger checks source metadata and finds that the currency every rollout uses should be EUR. (3) Adjudication in a fresh context combines the evidence into a revision plan, and delivery produces the revised report with its verification record.

entropy

$$\pi_p(v) = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[v_{i,p} = v], \quad H(\pi_p) = - \sum_{v \in \mathcal{V}_p} \pi_p(v) \log \pi_p(v). \quad (1)$$

A claim is a *consensus* claim when $H(\pi_p) = 0$ and a *disputed* claim when $H(\pi_p) > 0$. The entropy measures variation across candidates only: unanimity can hold even when every candidate is incorrect.

Empirical observations. On ten-rollout pools of Claude Opus 4.8 on APEX-Agents, whose rubrics permit claim-level grading (details in Appendix A), 34% of consensus values are judged incorrect, while 74% of disputed claims contain a correct candidate and the most frequent value is correct in only 47% (Figure 2). These statistics motivate two complementary checking tasks. For disputed claims, the verifier seeks evidence that distinguishes the exposed alternatives, while allowing that every candidate may be incorrect. For consensus claims, it must first propose how the consensus value could be wrong and then test those proposals against the environment. Proposing failures requires knowledge of how artifacts fail, which the pool does not supply. Section 3 develops a harness for both tasks.

3. VERIHARNESS

VERIHARNESS implements the two checking tasks of Section 2 with the same model as the generator. As Figure 3 illustrates, a disagreement resolver and a consensus challenger use shared tools and verification skills to produce evidence records. Adjudication in a fresh context turns both records into a selection and revision plan for the delivered artifact. Appendix B discusses why the same model can detect errors made during generation.

3.1. A General-Purpose Verification Harness

An agent harness $\mathcal{H}_G$ supplies the tools, memory, and operating loop that turn a frozen model M into the generator, $\text{Generator} = (M, \mathcal{H}_G)$. VERIHARNES applies the same construction to verification:

$$\text{Verifier} = (M, \mathcal{H}_V), \quad \mathcal{H}_V = (\mathcal{W}, \mathcal{T}, \Pi, \mathcal{S}), \quad (2)$$

where $\mathcal{W}$ is the workspace, $\mathcal{T}$ the evidence tools, Π the verification protocol, and $\mathcal{S}$ the skill library. Table 5 summarizes how they provide access, organize checking, and supply reusable knowledge.

Workspace and tools. The workspace exposes each rollout’s artifact d_i , final state w_i , and trace τ_i as files alongside the task description s and environment $\mathcal{E}$. The verifier can trace a claim to its source or computation, recompute quantities, and execute code to obtain evidence $\mathcal{E}(\chi)$ for a check χ .

Verification protocol. The protocol Π assigns the two checking tasks to separate model contexts, followed by adjudication and delivery. The program provides the contexts and information access; the model chooses which claims to examine, which checks to run, and when to stop. Because the workspace is file-based and the protocol does not depend on the artifact type, the same harness applies to reports, spreadsheets, and code; only the skills hold artifact-specific knowledge.

Verification skills. A skill describes a reusable failure mode and its checking procedure in a short text with an optional script. Tools perform a check, while skills identify what needs checking: a tool reads a financial table, whereas a skill directs the verifier to test whether a growth rate spans the years in its label. Skills exclude task-specific answers. Section 4 uses a human-authored library $\mathcal{S}$, listed in full with the tools in Appendix H; Section 5 studies how failure feedback can extend this library without changing the model or protocol.

3.2. Resolving Disagreement through Evidence

The disagreement resolver compares artifacts to locate a disputed claim p and candidate values $\mathcal{V}_p$, which it traces to their sources and computations. It chooses the check that best distinguishes them. Verification skills guide these choices, for example the source-version check in Figure 3 and checks of competing interpretations of a workbook convention. Let b_p be its current belief over the correct value given the evidence $e_{<k}$ gathered before step k , distinct from the empirical frequencies π_p in Section 2. The resolver prefers checks with large expected information gain (Lindley, 1956),

$$\text{Gain}(\chi) = \mathbb{E}[H(b_p) - H(b_p(\cdot \mid \mathcal{E}(\chi))) \mid e_{<k}], \quad (3)$$

which it judges qualitatively rather than computes. After running a check and obtaining evidence e , the resolver eliminates contradicted candidates:

$$\mathcal{V}_p \leftarrow \{v \in \mathcal{V}_p : v \text{ is consistent with } e\}. \quad (4)$$

The resolver closes a claim when the evidence settles it and stops when it judges that further checks are unlikely to change the outcome. If the evidence contradicts every candidate, it records a new value only when the evidence establishes one; otherwise the claim remains open. The resolver records all outcomes in the evidence record $L_\#$ for joint adjudication with the challenger’s findings.

3.3. Challenging Consensus

Consensus supplies no competing value to test, so the challenger identifies consensus claims, proposes how each could fail, and tests those possibilities against the environment. It prioritizes checks by how likely they are to expose a task-relevant error given the task, the pool, and the evidence so far,

$$\text{Priority}(\chi) = P(\chi \text{ exposes a failure of the consensus claim} \mid t, R, e_{<k}), \quad (5)$$

which it judges qualitatively rather than estimates numerically. The challenger examines three kinds of consensus claim:

- **Consensus values.** The challenger recomputes quantities from raw inputs and compares labels with source metadata, including the currency check in Figure 3.
- **Consensus readings.** The challenger tests a common interpretation against the file the task names, the period the source states, or the definition the workbook carries.
- **Consensus omissions.** The challenger checks the delivered artifacts against the task description for requirements that every rollout overlooked.

An omission must first be made visible as a verification target because it appears in no artifact. More generally, challenging consensus requires knowledge of how an apparently settled artifact can fail. Skills supply that knowledge: a sign-convention skill, for example, gives the challenger a concrete property to test despite unanimous values. The challenger records its findings in the evidence record $L_{=}$.

3.4. Adjudication and Evidence-Backed Revision

Adjudication. The investigations run in separate contexts so that disputed claims do not displace attention from consensus claims; Appendix F evaluates this separation. A fresh context of the same model then reviews both records with the task and rollouts, without inheriting either investigator’s conversational history. It can uphold, overturn, or leave findings unresolved. It outputs a base rollout, an evidence-supported revision plan, and a list of unresolved claims:

$$(\text{base}, \text{revision_plan}, \text{unresolved}) = A(L_{\neq}, L_{=}, t, R), \quad (6)$$

where $\text{base} \in R \cup \{\emptyset\}$ is the rollout to build on, revision_plan specifies the changes to make and their supporting evidence, and unresolved lists claims that the evidence has not settled. Adjudication selects the base with the fewest evidence-supported defects on the task’s substantive requirements, weighing findings by their importance to the requested deliverable. When no candidate provides a usable starting point, it sets $\text{base} = \emptyset$.

Delivery and verification record. In selection-only evaluation, the adjudicated $\text{base} \in R$ is returned unchanged. The full harness applies the revision plan and returns the resulting rollout with its verification record:

$$\begin{aligned} r^* &= \text{Apply}(\text{base}, \text{revision_plan}, \text{unresolved}), \\ C &= (L_{\neq}, L_{=}, \text{base}, \text{revision_plan}, \text{unresolved}). \end{aligned} \quad (7)$$

An empty revision plan leaves the base unchanged; a nonempty plan revises it, or produces a new artifact from the plan when $\text{base} = \emptyset$. Claims in unresolved stay marked as unsettled in C , and delivery presents both readings where the artifact format allows.

Each evidence record links a claim, check, evidence, and verdict, making changes traceable and preserving open issues. Together with grader feedback on development tasks, these records support skill development in Section 5. Appendix C gives the complete procedure and delivery rules.

4. Experiments

4.1. Setup

Evaluation scope. We evaluate on five recent long-horizon workspace benchmarks spanning professional documents, spreadsheets, code, and multi-file tasks: APEX-Agents (version 1.0) (Vidgen et al.,

2026), Workspace-Bench Lite (Tang et al., 2026), WorkBuddy Bench (WorkBuddy Team et al., 2026), SpreadsheetBench 2 (Zhu et al., 2026), and JobBench (Li et al., 2026b). Table 1 summarizes their scale, artifacts, and graders. Appendix D provides further benchmark details.

Table 1 | **Benchmarks.** Tasks is the number of tasks in our pools; every task has $N = 10$ rollouts per model.

Benchmark	Tasks	Domain	Artifact	Grader
APEX-Agents (v1.0)	480	banking, law, consulting	memos, models, answers	expert rubric
Workspace-Bench Lite	100	file-heavy workspaces	documents and files	rubric judge
WorkBuddy Bench	200	code, office, web	patches and office files	composite verifier
SpreadsheetBench 2	321	business spreadsheets	workbooks	recompute, compare
JobBench	65	35 white-collar occupations	multi-file artifacts	rubric judge

Same-model comparison. We use Gemini 3.5 Flash and Claude Opus 4.8 as both generator and verifier, with thinking level *high*. For every task and model, all methods receive the same frozen pool of $N = 10$ rollouts and the same model. Outputs are scored by the benchmark graders, which are never available to the verifier. Appendix D gives the task, runtime, and grading details.

Baselines. We compare VERIHARNESS with majority voting, best-of- N judging, a pairwise tournament, and the most recent LLM-as-a-Verifier (Kwok et al., 2026). We also implement an *agentic verifier* (*env. access*) given the same workspace and evidence tools as our verifier but no VeriHarness protocol or skills. VERIHARNESS (select) returns the base chosen by adjudication, and VERIHARNESS also applies the evidence-backed revision plan. For the latter, we add aggregation of the pool into a new artifact (Lee et al., 2026b; Wang et al., 2025a), which also produces an artifact absent from the pool. We also include an *agentic verifier + revision* baseline: the same model checks the pool against the workspace, selects a base, and revises it using the same evidence tools. The grader-informed selection oracle bounds selection from the fixed pool. We report each benchmark’s native score over three independent verifier runs. Appendix D defines the baselines, metrics, and revision protocol.

4.2. Results

VERIHARNESS outperforms all selection baselines across both models. Table 2 shows that it improves the single-rollout average by 4.4 points with Flash and 4.1 points with Opus, exceeds every baseline in all ten model–benchmark settings, and gains more than the cross-seed spread in every cell. Majority voting and best-of- N judging yield only modest average gains on these long-horizon workspace tasks. The consistent advantage over methods that judge only the candidate pool supports the value of checking claims against environmental evidence, and the agentic verifier with the same environment access but no protocol or skills recovers only about half of the harness’s gain, so access alone is not sufficient.

The gain transfers to existing agent harnesses. Running the protocol inside Gemini CLI, Claude Code, or Codex instead of our own implementation preserves most of the selection improvement: with Flash, both CLIs match our implementation within two points on every benchmark, and with Opus the CLIs recover most of the gain on Workspace-Bench Lite and WorkBuddy Bench and about half of it on APEX-Agents, but fall back to the single-rollout level on JobBench, as does Claude Code on SpreadsheetBench 2. This result shows that the verification protocol and skills are not tied to one agent implementation; Appendix G discusses why we still provide a dedicated one.

Evidence-backed revision adds value beyond selection. Applying the revision plan improves all ten model–benchmark settings at the cost of one editing pass per task (Appendix K), and raises the average gain over a single rollout to 6.2 points with Flash and 6.4 points with Opus, with the largest

added gain on APEX-Agents with Opus and on Workspace-Bench Lite with Flash. Aggregating the pool into a new artifact without evidence checks stays below the harness on every benchmark, so the revision gain comes from the evidence records rather than from rewriting itself.

Table 2 | **Main results.** Three-seed mean score on each benchmark’s own scale, with the cross-seed standard deviation in gray; Avg is the unweighted mean over the five benchmarks, and gains are over the single rollout. Within each model, bold marks the best score in the primary selection comparison and, separately, among the methods with revision; CLI variants are reported separately, followed by methods that produce new artifacts. The single rollout is a reference baseline. The selection oracle takes the highest-scoring rollout per task under the benchmark grader and bounds selection from the fixed pool; it does not bound revision. SB-2: SpreadsheetBench 2; WSB: Workspace-Bench Lite.

Method	Avg	APEX	WSB	WorkBuddy	SB-2	JobBench
Gemini 3.5 Flash						
Single rollout	47.2	48.1	56.5	73.2	27.3	30.9
Majority voting	47.5	47.3 ± 0.4	57.0 ± 1.8	74.0 ± 1.0	27.0 ± 1.3	32.4 ± 0.5
Best-of-N with judge	47.3	47.2 ± 0.4	57.0 ± 2.1	73.6 ± 1.2	27.7 ± 0.3	30.9 ± 0.9
Pairwise tournament	49.1	49.2 ± 1.4	59.6 ± 1.0	74.8 ± 0.8	30.4 ± 0.7	31.4 ± 1.5
LLM-as-a-Verifier	49.5	49.8 ± 0.5	60.4 ± 0.2	75.3 ± 0.5	30.2 ± 0.5	31.9 ± 1.2
Agentic verifier (env. access)	49.4	49.0 ± 0.8	60.1 ± 1.3	75.1 ± 0.9	30.9 ± 1.1	31.8 ± 1.4
VERIHARNESS (select)	51.6	53.3 ± 1.2	62.0 ± 0.9	77.8 ± 1.4	31.5 ± 2.4	33.5 ± 1.8
gain over single rollout	+4.4	+5.2	+5.5	+4.6	+4.2	+2.6
<i>Deployed in existing agent CLIs</i>						
VERIHARNESS in Gemini CLI	51.5	51.3 ± 1.1	63.2 ± 1.1	77.6 ± 0.8	31.5 ± 1.0	33.8 ± 1.2
VERIHARNESS in Codex	51.4	51.5 ± 1.1	63.8 ± 1.1	75.8 ± 1.1	30.5 ± 1.1	35.6 ± 1.3
Selection oracle	62.7	67.3	70.3	87.7	37.4	51.0
<i>With revision</i>						
Aggregation over the pool	49.9	49.6 ± 1.0	60.9 ± 1.5	74.4 ± 1.1	29.0 ± 1.2	35.6 ± 1.6
Agentic verifier + revision	50.7	50.5 ± 1.0	62.0 ± 1.5	76.0 ± 1.0	31.0 ± 1.2	34.0 ± 1.5
VERIHARNESS	53.4	54.8 ± 1.0	65.4 ± 1.0	78.2 ± 0.2	32.2 ± 0.6	36.2 ± 1.1
gain over single rollout	+6.2	+6.7	+8.9	+5.0	+4.9	+5.3
Claude Opus 4.8						
Single rollout	49.6	35.8	60.5	79.0	30.1	42.8
Majority voting	50.4	38.0 ± 0.2	62.0 ± 0.7	79.3 ± 0.7	29.5 ± 0.5	43.3 ± 1.5
Best-of-N with judge	50.6	38.4 ± 0.4	62.8 ± 0.3	79.2 ± 1.1	30.5 ± 0.0	42.3 ± 1.9
Pairwise tournament	51.3	38.4 ± 1.1	63.1 ± 0.7	80.5 ± 1.4	31.4 ± 0.7	42.9 ± 0.9
LLM-as-a-Verifier	51.2	36.7 ± 0.8	62.4 ± 0.2	81.7 ± 0.3	30.2 ± 0.3	45.0 ± 1.8
Agentic verifier (env. access)	51.6	38.1 ± 0.9	63.4 ± 0.6	80.9 ± 0.7	31.8 ± 0.8	43.9 ± 1.3
VERIHARNESS (select)	53.7	41.2 ± 0.7	65.0 ± 0.5	83.0 ± 0.4	33.4 ± 1.2	46.1 ± 0.8
gain over single rollout	+4.1	+5.4	+4.5	+4.0	+3.3	+3.3
<i>Deployed in existing agent CLIs</i>						
VERIHARNESS in Claude Code	51.7	39.0 ± 1.1	64.9 ± 1.3	81.5 ± 0.9	30.9 ± 1.0	42.4 ± 1.4
VERIHARNESS in Codex	52.2	38.4 ± 1.1	63.9 ± 1.2	81.8 ± 0.9	33.1 ± 1.0	43.6 ± 1.5
Selection oracle	63.1	53.8	72.4	89.4	39.6	60.4
<i>With revision</i>						
Aggregation over the pool	52.3	40.2 ± 1.1	63.5 ± 0.8	80.8 ± 0.9	30.9 ± 0.7	45.9 ± 1.4
Agentic verifier + revision	53.1	42.0 ± 1.0	64.5 ± 0.8	82.0 ± 0.7	32.0 ± 1.0	45.0 ± 1.3
VERIHARNESS	56.1	47.5 ± 0.2	67.3 ± 0.2	83.7 ± 0.7	34.3 ± 0.2	47.6 ± 0.5
gain over single rollout	+6.4	+11.7	+6.8	+4.7	+4.2	+4.8



Figure 4 | **Evolved libraries improve held-out scores.** (a) Held-out scores of the four libraries. A: empty; B: human-authored; C: evolved from A; D: evolved from B. (b) Held-out scores recorded each round for C and D, with A and B as dashed lines; end labels give the final-library scores of (a). Filled markers after round 0 indicate a new library retained using development scores; hollow markers indicate that the current library is unchanged.

5. VeriHarness Can Self-Evolve

In this section, we study whether the verification skills can accumulate from failure feedback. The skill library S supplies reusable knowledge of what to check and how to revise an artifact. Section 4 uses a human-authored library. Here we let the same verifier model improve the library from development-task failures, starting either from an empty library or from the human-authored library. The model, verification protocol Π , and general tools remain fixed.

The learning loop. We split the eligible tasks of each benchmark into a development set and a held-out test set at a ratio of about 3:1. In each round, the verifier model reads development failures together with per-item grader feedback and proposes candidate libraries. The candidate with the highest development score replaces the current library if it matches or exceeds that library’s score. Held-out scores are recorded each round but never used for selection, and verification on held-out tasks sees no reference answers or grading rubrics. Appendix I details the split, the failure packages, the filtering of task-specific content, and the selection rule.

Four conditions. We compare **A**, an empty library; **B**, the frozen human-authored library; **C**, the final library evolved from A; and **D**, the final library evolved from B. All four use the full harness, including evidence-backed revision, with Claude Opus 4.8 on APEX-Agents and SpreadsheetBench 2. These experiments use held-out subsets, so their scores are distinct from the full-benchmark results in Table 2. A versus C measures learning from failures, B versus C compares learned and human-authored skills, and B versus D measures improvement beyond human initialization.

Skills learned from empty outperform the human-authored library. Relative to A, C improves the held-out score by 11.0 points on APEX-Agents and 5.7 points on SpreadsheetBench 2 (Figure 4a). C also exceeds B on both benchmarks. With the model and protocol fixed, these comparisons show that failure feedback can improve the reusable skills that guide checking and revision.

Human initialization improves final performance and accelerates progress. Evolving from B yields the highest final score on both benchmarks, adding 6.8 points on APEX-Agents and 3.7 points on SpreadsheetBench 2 over B. Figure 4b shows that D reaches C’s final trajectory score earlier on both benchmarks, with a larger reduction in rounds on APEX-Agents. Appendix I gives the round-by-round comparison and distinguishes progress in rounds from compute efficiency.

Examples of learned skills. The learned libraries give the verifier specific instructions for checking an artifact. An APEX-Agents skill instructs it to list every item the task requests, compare how the candidate answers address each item, and check the supporting evidence before revising the answer. A SpreadsheetBench 2 skill instructs it to read the period in a growth-rate label and check that the formula uses the corresponding start and end columns. These examples illustrate the reusable checking procedures added through evolution. Appendix I gives further examples.

6. Analysis

We ask four questions about the harness: which parts of the harness carry the gain, on which tasks selection gains, what happens when the verifier faces a unanimous pool, and how the evolved libraries differ from the human-authored library.

The disagreement resolver and the consensus challenger recover different errors. Table 4 in Appendix F runs the full harness with one part removed. With Claude Opus 4.8 the resolver alone raises the five-benchmark average from 49.6 to 54.3 and the challenger alone to 52.6, and the two together reach 56.1, so their gains are complementary. The resolver’s share is largest on APEX-Agents, where disputed claims are dense, and the two are closest on SpreadsheetBench 2 and JobBench, where shared errors in units, sign conventions, and omitted requirements make up more of the gap. Merging the two investigations into one context costs 0.8 points, which supports running them in separate contexts. The same ordering holds with Gemini 3.5 Flash.

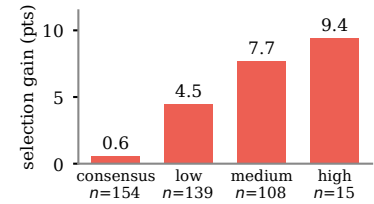


Figure 5 | **Selection gains where the pool disagrees.** Selection score minus pool mean on APEX-Agents, Claude Opus 4.8, by the claim entropy of the pool.

Selection gains come from the pools that disagree. Following Figure 2, we group the APEX-Agents tasks of Claude Opus 4.8 by the claim entropy of their pools. On the 154 consensus pools, where the ten rollouts give the same answer, the harness’s selection scores only 0.6 points above the pool mean, while on the 262 disputed pools it scores 6.1 points above. Within the disputed pools the gain grows with entropy, from 4.5 points on low-entropy pools to 7.7 on medium-entropy pools (Figure 5). Section 2 observed that disagreement exposes correct alternatives, and the harness turns these alternatives into score.

Challenging consensus generally does no harm, and its blind spot calls for more prior knowledge. On APEX-Agents we match the challenger’s verdicts on unanimous claims to the rubric items they concern (Table 7 in Appendix J). The challenger is generally safe: it never refuted a claim that every rollout had right. The shared errors it left standing are mostly a matter of where it looked: in about 70% of them it checked an intermediate result that was correct, while the error sat in a later step. The verifier tends to stop where the rollouts stopped, and telling it where else to look requires prior knowledge of how artifacts fail, which the skill library supplies.

The evolved libraries are more specific and learn the verifier’s own blind spot. We manually compare the 95 checks in the four evolved libraries with the human-authored library: 5 restate a human check, 48 make a human check concrete with a script, a constant, or a document type, and 42 are new. Evolved skills are thus usually more specific than human-authored skills: about half of the evolved checks turn a general human check into a procedure for one kind of artifact. The new checks also cover the blind spot for consensus. The proposer reads each development failure together with the verifier’s evidence record and the grader’s verdicts, so it can see where the verifier confirmed a shared value that the grader marked wrong. From such cases, the libraries evolved on APEX-Agents and on SpreadsheetBench 2, in separate runs, both instruct the verifier not to treat a recomputation

that matches the pool as confirmation.

7. Related Work

Verifying agent outputs. Verification of model outputs began as judgment: a model is prompted as a judge (Zheng et al., 2023), or a verifier or process reward model is trained on labeled data (Cobbe et al., 2021; Lightman et al., 2024; Zhang et al., 2025). Recent work scales verification at test time through finer scores, repeated evaluation, and decomposition into checkable criteria (Kwok et al., 2026; Lifshitz et al., 2025; Wan et al., 2026; Zeng et al., 2026a; Zhao et al., 2026), and a newer line lets the verifier act, by inspecting the workspace against a requirement list, gathering repository evidence, or calling tools under reinforcement learning (Yuan et al., 2026; Zeng et al., 2026b; Zhang et al., 2026d; Zhuge et al., 2025). Three findings limit verification by the same model: a model cannot correct its own reasoning without external feedback (Huang et al., 2024), its self-checks are mostly confirmatory (Long et al., 2026), and its errors are correlated with those of a judge from the same model family (Goel et al., 2025). The methods above train the verifier or check a single output against given criteria. VERIHARNESS is training-free and uses the same model as the generator; it takes its targets from the disagreement and the consensus among rollouts, settles them with evidence from the environment, and returns a better rollout instead of a score.

Test-time scaling for agents. Repeated sampling raises the chance that some rollout is correct, but voting and reward models often fail to identify it (Brown et al., 2024; Snell et al., 2025). Studies of agents reach the same conclusion, with parallel scaling stalling at a verification gap and sequential scaling at a context ceiling (Li et al., 2026a; Zhu et al., 2025). Proposed remedies operate on the rollouts themselves: they vote over them, compare them in tournaments, or fuse them with an aggregating agent (Chen et al., 2023; Kim et al., 2026; Lee et al., 2026b; Wang et al., 2025a, 2023; Zhao et al., 2025). In test-time scaling, external evidence has mainly been used for code, through generated tests or distinguishing inputs (Ehrlich et al., 2025; Li et al., 2025). Debate also addresses the setting where the judge is no stronger than the debaters, but it relies on argument between models rather than evidence from the environment (Irving et al., 2018; Khan et al., 2024). VERIHARNESS brings evidence acquisition to tasks without executable tests, and it uses the rollouts to decide what to check.

Harnesses and self-evolution. The agent harness is now a recognized layer of engineering around a frozen model (Anthropic, 2024; OpenAI, 2026; Wei, 2026), and Huang et al. (2026) extend it to the environment side. A recent line evolves the harness itself from execution traces, at test time or offline (Lee et al., 2026a; Lin et al., 2026; Nie et al., 2026; Zhang et al., 2026a,c), and a longer tradition grows skill and experience libraries without touching the weights (Gao et al., 2026; Ouyang et al., 2026; Wang et al., 2024, 2025b; Zhang et al., 2026e), with Zhang et al. (2026b) making verification itself an evolving skill. Wang et al. (2026) find that evolved harnesses often fail to beat budget-matched test-time scaling and overfit when development and evaluation sets coincide. VERIHARNESS evolves the verifier’s skills rather than the generator’s scaffold: candidates are admitted on a development set, evaluated on held-out tasks, and need no ground truth at deployment.

8. Conclusion

We presented VERIHARNESS, a verification harness that uses the generator’s own model to resolve disagreement and challenge consensus through environmental evidence. Across five workspace benchmarks and two models, it achieves the highest selection scores among the evaluated baselines, while evidence-backed revision improves average performance over a single rollout by more than six points with each model. We further show that skills evolved from development feedback outperform

the human-authored library on held-out tasks, and human initialization further improves final performance. Verification capability therefore does not have to wait for a stronger model. It can be built around the model that exists, from the structure of its own rollouts, the evidence in its environment, and its accumulated experience. Beyond test-time use, the verification record links every claim to its check, evidence, and verdict, and thus provides a claim-level signal that could supervise the generator; turning these records into training data for long-horizon agents is a natural next step.

Limitations

Cost of the rollout pool. VERIHARNESS verifies from a pool of rollouts, and the reported results use ten per task. This is the standard budget of test-time scaling: every baseline in Table 2 receives the same ten rollouts, so the comparison is made at equal generation cost, and Appendix K reports the cost of verification itself.

No score for individual rollouts. The harness delivers an artifact and a verification record, and it does not assign a scalar score to each rollout. A long-horizon artifact contains many claims that can be right or wrong independently, and the harness judges them claim by claim against evidence. Turning these judgments into a calibrated rollout-level score, for example as a reward signal, is left to future work.

Same-model setting. We restrict our study to a verifier that is the same model as the generator. A stronger verifier may raise the scores further, but its gain would mix the contribution of the harness with the capability gap between the two models, and separating them is outside the scope of this paper.

Latency. Verification is a multi-turn investigation and adds wall-clock time to a task. We target professional deliverables such as reports, workbooks, and patches, where quality matters more than response time. Deep research systems make the same trade and run for many minutes to return a better answer.

References

- Anthropic. Building effective AI agents. Anthropic engineering blog, published 19 December 2024, 2024. URL <https://www.anthropic.com/research/building-effective-agents>.
- S. R. Bowman, J. Hyun, E. Perez, E. Chen, C. Pettit, S. Heiner, K. Lukošiušė, A. Askell, A. Jones, A. Chen, A. Goldie, A. Mirhoseini, C. McKinnon, C. Olah, D. Amodei, D. Amodei, D. Drain, D. Li, E. Tran-Johnson, J. Kernion, J. Kerr, J. Mueller, J. Ladish, J. Landau, K. Ndousse, L. Lovitt, N. Elhage, N. Schiefer, N. Joseph, N. Mercado, N. DasSarma, R. Larson, S. McCandlish, S. Kundu, S. Johnston, S. Kravec, S. E. Showk, S. Fort, T. Telleen-Lawton, T. Brown, T. Henighan, T. Hume, Y. Bai, Z. Hatfield-Dodds, B. Mann, and J. Kaplan. Measuring progress on scalable oversight for large language models, 2022. URL <https://arxiv.org/abs/2211.03540>.
- B. Brown, J. Juravsky, R. Ehrlich, R. Clark, Q. V. Le, C. Ré, and A. Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling, 2024. URL <https://arxiv.org/abs/2407.21787>.
- C. Burns, P. Izmailov, J. H. Kirchner, B. Baker, L. Gao, L. Aschenbrenner, Y. Chen, A. Ecoffet, M. Joglekar, J. Leike, I. Sutskever, and J. Wu. Weak-to-strong generalization: Eliciting strong capabilities with weak supervision. In *Proceedings of the 41st International Conference on Machine Learning*,

- volume 235 of *Proceedings of Machine Learning Research*, pages 4971–5012. PMLR, 2024. URL <https://proceedings.mlr.press/v235/burns24b.html>.
- X. Chen, R. Aksitov, U. Alon, J. Ren, K. Xiao, P. Yin, S. Prakash, C. Sutton, X. Wang, and D. Zhou. Universal self-consistency for large language model generation, 2023. URL <https://arxiv.org/abs/2311.17311>.
- K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman. Training verifiers to solve math word problems, 2021. URL <https://arxiv.org/abs/2110.14168>.
- R. Ehrlich, B. Brown, J. Juravsky, R. Clark, C. Ré, and A. Mirhoseini. CodeMonkeys: Scaling test-time compute for software engineering, 2025. URL <https://arxiv.org/abs/2501.14723>.
- H.-a. Gao, J. Geng, W. Hua, M. Hu, X. Juan, H. Liu, S. Liu, J. Qiu, X. Qi, Q. Ren, Y. Wu, H. WANG, H. Xiao, Y. Zhou, S. Zhang, J. Zhang, J. Xiang, Y. Fang, Q. Zhao, D. Liu, C. Qian, Z. Wang, M. Hu, H. Wang, Q. Wu, H. Ji, and M. Wang. A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence. *Transactions on Machine Learning Research*, 2026. ISSN 2835-8856. URL <https://openreview.net/forum?id=CTr3bovS5F>. Survey Certification.
- S. Goel, J. Strüder, I. A. Auzina, K. K. Chandra, P. Kumaraguru, D. Kiela, A. Prabhu, M. Bethge, and J. Geiping. Great models think alike and this undermines AI oversight. In A. Singh, M. Fazel, D. Hsu, S. Lacoste-Julien, F. Berkenkamp, T. Maharaj, K. Wagstaff, and J. Zhu, editors, *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 19621–19678. PMLR, 13–19 Jul 2025. URL <https://proceedings.mlr.press/v267/goel25b.html>.
- C. Huang, Z. Wang, R. Han, J. Yan, Y. Chen, Z. CuiZhu, K. Jiang, P. Xia, H. Yu, Y. Zhuang, Y. Ming, J. Pan, B. D. Mishra, J. Huang, B. Gokturk, T. Pfister, and C.-Y. Lee. EnvHarness: Awakening static worlds for agent learning, 2026. URL <https://arxiv.org/abs/2608.19880>.
- J. Huang, S. Gu, L. Hou, Y. Wu, X. Wang, H. Yu, and J. Han. Large language models can self-improve. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 1051–1068. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.emnlp-main.67. URL <https://aclanthology.org/2023.emnlp-main.67/>.
- J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou. Large language models cannot self-correct reasoning yet. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=lkmD3fKBPQ>.
- G. Irving, P. Christiano, and D. Amodei. AI safety via debate, 2018. URL <https://arxiv.org/abs/1805.00899>.
- A. Khan, J. Hughes, D. Valentine, L. Ruis, K. Sachan, A. Radhakrishnan, E. Grefenstette, S. R. Bowman, T. Rocktäschel, and E. Perez. Debating with more persuasive LLMs leads to more truthful answers. In R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett, and F. Berkenkamp, editors, *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 23662–23733. PMLR, 21–27 Jul 2024. URL <https://proceedings.mlr.press/v235/khan24a.html>.
- J. Kim, W. Yang, K. Niu, H. Zhang, Y. Zhu, E. Helenowski, R. Silva, Z. Chen, S. Iyer, M. Zaheer, D. Fried, H. Hajishirzi, S. Arora, R. Salakhutdinov, G. Synnaeve, and A. Goyal. Scaling test-time

- compute for agentic coding. In *Third Conference on Language Modeling*, 2026. URL <https://openreview.net/forum?id=J9kWWkRzUM>.
- J. Kwok, S. Li, P. Atreya, Y. Liu, Y. Jiang, C. Finn, M. Pavone, I. Stoica, and A. Mirhoseini. LLM-as-a-verifier: A general-purpose verification framework, 2026. URL <https://arxiv.org/abs/2607.05391>.
- Y. Lee, R. S. Nair, Q. Zhang, K. Lee, O. Khattab, and C. Finn. Meta-harness: End-to-end optimization of model harnesses. In *Third Conference on Language Modeling*, 2026a. URL <https://openreview.net/forum?id=tmb0UyFx3R>.
- Y. Lee, H. Yen, X. Ye, and D. Chen. Agentic aggregation for parallel scaling of long-horizon agentic tasks. In *Third Conference on Language Modeling*, 2026b. URL <https://openreview.net/forum?id=YF0F93vRnj>.
- D. Li, S. Cao, C. Cao, X. Li, S. Tan, K. Keutzer, J. Xing, J. E. Gonzalez, and I. Stoica. S*: Test time scaling for code generation. In C. Christodoulopoulos, T. Chakraborty, C. Rose, and V. Peng, editors, *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 15964–15978, Suzhou, China, Nov. 2025. Association for Computational Linguistics. ISBN 979-8-89176-335-7. doi: 10.18653/v1/2025.findings-emnlp.865. URL <https://aclanthology.org/2025.findings-emnlp.865/>.
- X. Li, R. Ming, P. Setlur, A. Paladugu, A. Tang, H. Kang, S. Shao, R. Jin, and C. Xiong. Benchmark test-time scaling of general LLM agents, 2026a. URL <https://arxiv.org/abs/2602.18998>.
- Y. Li, Y. Feng, Z. Xu, Z. Ma, K. Zheng, F. Jiang, X. Sun, R. Shao, Z. Chen, Y. Huang, X. Han, B. Lee, K. Xu, S. Zeng, H. Hua, X. Zhang, B. Alomair, R. Krishna, L. Zettlemoyer, P. W. Koh, B. Ramasubramanian, L. Niu, X. Yue, and R. Poovendran. Jobbench: Aligning agent work with human will, 2026b. URL <https://arxiv.org/abs/2605.26329>.
- S. Lifshitz, S. A. McIlraith, and Y. Du. Multi-agent verification: Scaling test-time compute with multiple verifiers. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=LriQ3NY9uL>.
- H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe. Let’s verify step by step. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=v8L0pN6E0i>.
- J. Lin, S. Liu, C. Pan, L. Lin, S. Dou, Z. Xi, X. Huang, H. Yan, Z. Han, T. Gui, and Y.-G. Jiang. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses, 2026. URL <https://arxiv.org/abs/2604.25850>.
- D. V. Lindley. On a measure of the information provided by an experiment. *The Annals of Mathematical Statistics*, 27(4):986–1005, 1956. doi: 10.1214/aoms/1177728069. URL <https://projecteuclid.org/journals/annals-of-mathematical-statistics/volume-27/issue-4/On-a-Measure-of-the-Information-Provided-by-an-Experiment/10.1214/aoms/1177728069.full>.
- Q. Long, K. J. Jiang, J. Chen, X. Guo, L. Gan, and W. Wang. Self-verification dilemma: Experience-driven suppression of overused checking in LLM reasoning, 2026. URL <https://arxiv.org/abs/2602.03485>.
- J. Nie, Y. Zhang, J. Song, Q. Cai, D. Yu, Y. Guo, X. Tian, and B. Han. TTHE: Test-time harness evolution, 2026. URL <https://arxiv.org/abs/2607.08124>.

- OpenAI. Codex as a platform: Build on the open agent harness. OpenAI Developers blog, published 19 August 2026, 2026. URL <https://developers.openai.com/blog/codex-as-a-platform>.
- S. Ouyang, J. Yan, I.-H. Hsu, Y. Chen, K. Jiang, Z. Wang, R. Han, L. Le, S. Daruki, X. Tang, V. Tirumalashetty, G. Lee, M. Rofouei, H. Lin, J. Han, C.-Y. Lee, and T. Pfister. ReasoningBank: Scaling agent self-evolving with reasoning memory. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=jL7fwchScm>.
- C. V. Snell, J. Lee, K. Xu, and A. Kumar. Scaling LLM test-time compute optimally can be more effective than scaling parameters for reasoning. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=4FWAwZtd2n>.
- Z. Tang, X. Zhou, Y. Liu, L. Li, Y. Wu, W. Wang, H. Huang, W. Zhou, J. Zhou, J. Song, S. Yu, J. Wang, Z. Zhou, H. Zhou, Y. Lv, J. Li, J. Liu, R. Chen, C. Liu, G. Li, J. Kang, and F. Wu. Workspace-bench 1.0: Benchmarking ai agents on workspace tasks with large-scale file dependencies, 2026. URL <https://arxiv.org/abs/2605.03596>.
- B. Vidgen, A. Mann, A. Fennelly, J. W. Stanly, L. Rothman, M. Burstein, J. Benchek, D. Ostrofsky, A. Ravichandran, D. Sur, N. Venugopal, A. Hsia, I. Robinson, C. Huang, O. Varones, D. Khan, M. Haines, A. Bridges, J. Boyle, K. Twist, Z. Richards, C. Mahapatra, B. Foody, and O. Nitski. Apex-agents, 2026. URL <https://arxiv.org/abs/2601.14242>.
- Y. Wan, T. Fang, Z. LI, Y. Huo, W. Wang, H. Mi, D. Yu, and M. R. Lyu. Inference-time scaling of verification: Self-evolving deep research agents via test-time rubric-guided verification. In M. Liakata, V. P. Moreira, J. Zhang, and D. Jurgens, editors, *Findings of the Association for Computational Linguistics: ACL 2026*, pages 24822–24835, San Diego, California, United States, July 2026. Association for Computational Linguistics. ISBN 979-8-89176-395-1. doi: 10.18653/v1/2026.findings-acl.1243. URL <https://aclanthology.org/2026.findings-acl.1243/>.
- G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. URL <https://openreview.net/forum?id=ehfRiFOR3a>.
- J. Wang, J. WANG, B. Athiwaratkun, C. Zhang, and J. Zou. Mixture-of-agents enhances large language model capabilities. In *The Thirteenth International Conference on Learning Representations*, 2025a. URL <https://openreview.net/forum?id=h0ZfDIrj7T>.
- X. Wang, J. Wei, D. Schuurmans, Q. V. Le, E. H. Chi, S. Narang, A. Chowdhery, and D. Zhou. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=1PL1NIMMrw>.
- Y. Wang, H. Zhu, Z. Hu, Y. Yuan, Z. Chen, S. Senthil, H. Hajishirzi, Y. Tsvetkov, P. Dasigi, and T. Xiao. Rethinking the evaluation of harness evolution for agents, 2026. URL <https://arxiv.org/abs/2607.12227>.
- Z. Z. Wang, J. Mao, D. Fried, and G. Neubig. Agent workflow memory. In A. Singh, M. Fazel, D. Hsu, S. Lacoste-Julien, F. Berkenkamp, T. Maharaj, K. Wagstaff, and J. Zhu, editors, *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 63897–63911. PMLR, 13–19 Jul 2025b. URL <https://proceedings.mlr.press/v267/wang25bx.html>.
- H. Wei. Architectural design decisions in AI agent harnesses, 2026. URL <https://arxiv.org/abs/2604.18071>.

- WorkBuddy Team, S. Cai, S. Chen, X. Fei, Y. Mao, Z. Xu, Z. Lyu, Z. Shao, Y. Shi, S. Zhang, C. Qiu, L. Che, X. Zhao, F. Wu, K. Zhang, C. Zhu, Y. Qi, X. Liang, P. Dong, Y. Zhang, Y. Zhu, L. Jiang, X. Zhang, Z. Chu, A. Sang, Z. Feng, S. Nie, S. Wu, Y. Xu, X. Li, N. Yang, Z. Dong, H. Dong, Q. Lin, Y. Liu, Y. Wu, K. Li, and X. Sun. Tencent workbuddy bench: A multi-domain coding-agent benchmark with contamination-resistant task construction, 2026. URL <https://arxiv.org/abs/2607.20911>.
- R. You, H. Cai, C. Zhang, Q. Xu, M. Liu, T. Yu, Y. Li, and W. Li. Agent-as-a-judge, 2026. URL <https://arxiv.org/abs/2601.05111>.
- A. Yuan, Y. Nian, H. Zhang, Z. Su, and Y. Zhao. SEVA: Self-evolving verification agent with process reward for fact attribution, 2026. URL <https://arxiv.org/abs/2606.29713>.
- W. Zeng, K. He, C. Kuang, X. Li, and J. He. Pushing test-time scaling limits of deep search with asymmetric verification. In *The Fourteenth International Conference on Learning Representations*, 2026a. URL <https://openreview.net/forum?id=hxL4Uf9tR3>.
- W. Zeng, Y. Shi, X. Gu, C. Hu, C. Wang, Y. Cui, H. Zhou, M. Qi, J. Wangni, Z. Yu, S. Gao, K. Cai, and S. He. Dockerless: Environment-free program verifier for coding agents, 2026b. URL <https://arxiv.org/abs/2606.28436>.
- G. Zhang, L. Lu, F. Xie, K. Zhu, J. Wang, Z. Xie, Z. Yu, Z. Liu, Z. Sun, Q. Li, Y. Liao, H. Chang, X. Hu, Q. Ren, W. Zhou, C. Hu, Y. Deng, and S. Yan. JIT-agent: Scaling harness intelligence via just-in-time harness evolution, 2026a. URL <https://arxiv.org/abs/2608.25593>.
- H. Zhang, S. Fan, H. P. Zou, Y. Chen, Z. Wang, J. Zhou, C. Li, W.-C. Huang, Y. Yao, K. Zheng, X. Liu, X. Li, and P. S. Yu. CoEvoSkills: Self-evolving agent skills via co-evolutionary verification. In *Third Conference on Language Modeling*, 2026b. URL <https://openreview.net/forum?id=gQjmJIichQ>.
- H. Zhang, S. Zhang, K. Li, C. Zhang, Y. Chen, Y. Zhang, L. Bai, and S. Hu. Self-harness: Harnesses that improve themselves, 2026c. URL <https://arxiv.org/abs/2606.09498>.
- J. Zhang, Z. Fu, Z. Xi, W. Jing, M. Chai, W. He, G. Zhang, C. Fan, C. An, W. Chen, Z. Liu, H. Pan, D. Zhu, T. Gui, Q. Zhang, and X. Huang. AgentV-RL: Scaling reward modeling with agentic verifier. In M. Liakata, V. P. Moreira, J. Zhang, and D. Jurgens, editors, *Findings of the Association for Computational Linguistics: ACL 2026*, pages 23078–23100, San Diego, California, United States, July 2026d. Association for Computational Linguistics. ISBN 979-8-89176-395-1. doi: 10.18653/v1/2026.findings-acl.1156. URL <https://aclanthology.org/2026.findings-acl.1156/>.
- J. Zhang, S. Hu, C. Lu, R. T. Lange, and J. Clune. Darwin Gödel machine: Open-ended evolution of self-improving agents. In *The Fourteenth International Conference on Learning Representations*, 2026e. URL <https://openreview.net/forum?id=pUpzQZTvGY>.
- L. Zhang, A. Hosseini, H. Bansal, M. Kazemi, A. Kumar, and R. Agarwal. Generative verifiers: Reward modeling as next-token prediction. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=Ccwp4tFEtE>.
- E. Zhao, P. Awasthi, and S. Gollapudi. Sample, scrutinize and scale: Effective inference-time search by scaling verification. In A. Singh, M. Fazel, D. Hsu, S. Lacoste-Julien, F. Berkenkamp, T. Maharaj, K. Wagstaff, and J. Zhu, editors, *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 77272–77309. PMLR, 13–19 Jul 2025. URL <https://proceedings.mlr.press/v267/zhao25a.html>.

- J. X. Zhao, H. Chen, B. Hooi, and S.-K. Ng. FineVerify: Scaling test-time compute with fine-grained self-verification for agentic search, 2026. URL <https://arxiv.org/abs/2606.00660>.
- L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. Xing, H. Zhang, J. Gonzalez, and I. Stoica. Judging LLM-as-a-judge with MT-bench and chatbot arena. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 46595–46623. Curran Associates, Inc., 2023. doi: 10.52202/075280-2020. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/91f18a1287b398d378ef22505bf41832-Paper-Datasets_and_Benchmarks.pdf.
- J. Zhu, Y. Zhang, Z. Ma, B. Zhang, A. Schoepf, D. Woloch, P. Y. Wang, G. R. Yang, S. Jacob, S. Nagisetty, A. Chundru, J. Lin, S. Mateega, and J. Zhang. Spreadsheetbench 2: Evaluating agents on end-to-end business spreadsheet workflows, 2026. URL <https://arxiv.org/abs/2606.29955>.
- K. Zhu, H. Li, S. Wu, T. Xing, D. Ma, X. Tang, M. Liu, J. Yang, J. Liu, Y. E. Jiang, C. Zhang, C. Lin, J. Wang, G. Zhang, and W. Zhou. Scaling test-time compute for LLM agents, 2025. URL <https://arxiv.org/abs/2506.12928>.
- M. Zhuge, C. Zhao, D. R. Ashley, W. Wang, D. Khizbullin, Y. Xiong, Z. Liu, E. Chang, R. Krishnamoorthi, Y. Tian, Y. Shi, V. Chandra, and J. Schmidhuber. Agent-as-a-judge: Evaluate agents with agents. In A. Singh, M. Fazel, D. Hsu, S. Lacoste-Julien, F. Berkenkamp, T. Maharaj, K. Wagstaff, and J. Zhu, editors, *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 80569–80611. PMLR, 13–19 Jul 2025. URL <https://proceedings.mlr.press/v267/zhuge25a.html>.

Appendix Contents

A	Claim-Level Analysis Protocol.....	19
B	Why Same-Model Verification Can Improve Outputs.....	19
C	Verification Procedure and Delivery Details.....	19
D	Experimental Details	20
E	Released Rollout Pool.....	22
F	Ablation of Harness Components.....	22
G	Why a Dedicated Runtime	23
H	Tools and Skills	23
I	Self-Evolution Details.....	26
J	Why Shared Errors Survive	27
K	Compute Cost.....	27

A. Claim-Level Analysis Protocol

This appendix describes how the claim statistics of Figure 2 are computed. The analysis uses the ten-rollout pools of Claude Opus 4.8 on APEX-Agents and covers 434 tasks and 1,692 claims. It is an offline analysis that reads the benchmark’s rubrics, which the verifier never sees.

Claims and values. Each rubric criterion names one quantity or determination that the answer should state, and we treat each criterion as one claim p . Claude Opus 4.8 reads the ten answers at temperature zero and records the value $v_{i,p}$ that each rollout asserts for the criterion. Numbers count as the same value when they agree at the criterion’s precision, determinations count as the same value when they state the same conclusion, and a rollout that does not address the criterion receives the value *absent*. The grouped values give π_p and $H(\pi_p)$ in Eq. (1), which yields 917 consensus claims and 775 disputed claims.

Correctness. Correctness comes from the benchmark’s own grades, which mark every rollout as passing or failing each criterion. A value is correct when the majority of the rollouts asserting it pass the criterion. For consensus claims, we report how often the agreed value is correct. For disputed claims, we report how often at least one rollout passes the criterion and how often the most frequent value is correct, averaging over tied values.

B. Why Same-Model Verification Can Improve Outputs

With model parameters fixed, the verifier can still operate with different information, a narrower objective, and accumulated checking experience. Table 3 explains how these differences can help it detect errors made during generation.

Table 3 | **Sources of advantage in same-model verification.**

Source	How it helps verification
Exposed alternatives	Multiple rollouts make competing values and interpretations explicit. The verifier can investigate a concrete discrepancy that an individual generation attempt never encountered.
Focused checking	Given completed artifacts, the verifier can devote its attention to testing particular claims. Separate resolver and challenger contexts reserve attention for both disputed and shared claims.
Environmental evidence	A targeted source lookup or recomputation can reveal evidence omitted from the rollouts. The generator has the same tools; verification directs their use toward a specific claim whose validity is now in question.
Accumulated skills	Human-authored checks and procedures learned from development failures suggest what to examine, including requirements that every candidate omitted. This experience can guide later verification without changing model weights.

These advantages explain how verification can improve with a fixed model. Their value depends on finding informative checks: shared blind spots can persist when a required claim goes unexamined or the available evidence cannot settle it.

C. Verification Procedure and Delivery Details

Algorithm 1 summarizes the procedure described in Section 3. The program provides the workspace and model contexts; each context uses the same model to choose checks, interpret evidence, and

make the decisions assigned to it.

Algorithm 1 VERIHARNESS

Require: task $t = (s, \mathcal{E})$, rollouts R , model M , skills $\mathcal{S}$

- 1: expose workspace $\mathcal{W}$ containing R , tools $\mathcal{T}$, protocol Π , and skills $\mathcal{S}$ ▷ program
 - 2: open two contexts of the verifier that run concurrently and cannot see each other ▷ program
 - 3: **disagreement resolver context:** the verifier identifies the disputed claims
 - 4: **while** the verifier finds a disputed claim worth a check **do**
 - 5: choose a claim p and a check χ with large $\text{Gain}(\chi)$ (Eq. (3)); $e \leftarrow \mathcal{E}(\chi)$; eliminate by Eq. (4);
 record in $L_{\neq}$
 - 6: **end while**
 - 7: **consensus challenger context:** the verifier identifies the consensus claims
 - 8: **while** the verifier finds a consensus claim worth a challenge **do**
 - 9: choose a check χ with high $\text{Priority}(\chi)$ (Eq. (5)); $e \leftarrow \mathcal{E}(\chi)$; record in $L_{=}$ whether the claim
 holds
 - 10: **end while**
 - 11: open a fresh context of the verifier on $(L_{\neq}, L_{=}, t, R)$ ▷ program
 - 12: $(\text{base}, \text{revision_plan}, \text{unresolved}) \leftarrow A(L_{\neq}, L_{=}, t, R)$
 - 13: $r^* \leftarrow \text{Apply}(\text{base}, \text{revision_plan}, \text{unresolved})$ ▷ same context as adjudication
 - 14: $C \leftarrow (L_{\neq}, L_{=}, \text{base}, \text{revision_plan}, \text{unresolved})$
 - 15: **return** (r^*, C)
-

Delivery rules. When `revision_plan` is empty, delivery returns the selected base unchanged. Otherwise, it applies the evidence-supported changes to the base; when `base` = $\emptyset$, it rebuilds the artifact from the inputs and evidence. Claims in `unresolved` remain marked as unsettled in the verification record. For unresolved interpretations, where the artifact format admits it, delivery presents both readings with the base’s reading first; where the format has a single slot, it takes the reading preferred by adjudication.

D. Experimental Details

Benchmarks. All five benchmarks require an agent to produce an artifact from a workspace without exposing the grading rubric or reference answer. APEX-Agents (Vidgen et al., 2026) contains long-horizon investment-banking, legal, and consulting tasks in file-rich simulated environments. All our experiments use version 1.0 of the benchmark. Workspace-Bench (Tang et al., 2026) contains realistic workspaces with large file-dependency graphs; we use its 100-task Lite subset. WorkBuddy Bench (WorkBuddy Team et al., 2026) contains tasks derived from real commits and business scenarios; we evaluate its code, office, and web domains, with 80, 50, and 70 tasks, and do not run its security domain, whose tasks consistently trigger safety alerts from the model APIs. SpreadsheetBench 2 (Zhu et al., 2026) evaluates financial modeling, template completion, debugging, and visualization in large workbooks. JobBench (Li et al., 2026b) covers 35 white-collar occupations with heterogeneous reference files.

Models and rollout pools. We use Gemini 3.5 Flash and Claude Opus 4.8 with the thinking level set to high. For each benchmark and model, the generator runs in the benchmark’s own agent scaffold with its standard tools and instructions and produces $N = 10$ rollouts per task. The scaffolds are the APEX-Agents agent runner with its tool servers, opencode for JobBench, the SpreadsheetBench 2 agent with its shell and spreadsheet-viewing tools, and Claude Code for WorkBuddy Bench and Workspace-Bench Lite. Both models use the same scaffold on each benchmark. Each rollout includes

its artifact, final workspace state, and trace. We freeze these pools before verification, so every method operates on the same candidates.

Harness runtime. Our runtime is a thin layer over the open-source pi coding agent (MIT license), which provides the agent loop, tool calling, and session records; Claude is served through a local LiteLLM proxy. The verifier has file reading, search, and shell tools. Rollouts, task inputs, and the environment are mounted read-only in a sandbox with no network access; only the output directory is writable. Spreadsheets are also rendered as cell-level text for comparison, and evidence skills cover spreadsheets, documents, presentations, PDFs, document bundles, and code patches (Appendix H). The two investigations run concurrently in separate contexts of the same model, and neither reads the other’s record. Adjudication opens a fresh context that receives the two evidence records, and delivery continues in the adjudication context. The CLI rows of Table 2 run the same instruction and skills inside Gemini CLI 0.46.0, Claude Code 2.1.246, and Codex 0.149.1, using each CLI’s own tools and sandbox in place of our runtime.

Grading protocol. Each benchmark’s own grader scores the delivered artifacts; the verifier never accesses that grader. APEX-Agents and JobBench grade each rubric item with an LLM judge, for which we use Gemini 3 Flash. Workspace-Bench Lite uses its agent-as-a-judge, a Claude Code agent with Claude Opus 4.8. WorkBuddy Bench uses its composite verifier, which applies tests and rules in the code domain, rules and an LLM judge (Gemini 3.5 Flash) in the office domain, and the benchmark’s judge pipeline in the web domain. SpreadsheetBench 2 recalculates the delivered workbook and compares it cell by cell with the reference workbook. When the grader is a judge model, the base and delivered rollouts are scored in the same round. We report the archived base score plus their same-round difference, which controls for judge drift between rounds.

Baselines. Every method receives the same $N = 10$ rollouts and model. The *single rollout* score is the pool mean. *Majority voting* selects the rollout most consistent with the others, extending self-consistency (Wang et al., 2023) to open-ended artifacts. *Best-of- N with a judge* scores all rollouts against the task description in one call. The *pairwise tournament* repeatedly compares two rollouts and advances the winner. *LLM-as-a-Verifier* (Kwok et al., 2026) decomposes the task into criteria and ranks candidates through per-criterion pairwise comparisons. The *agentic verifier (env. access)* runs the best-of- N judge as a tool-using agent in the same sandbox as our verifier, with the same file, search, and shell tools, but with a single scoring instruction and no protocol or skills. The *selection oracle* selects the rollout with the highest grader score for each task and serves only as an upper bound on selection from the fixed pool. For the full harness, *aggregation over the pool* asks the model to write a new artifact from the N rollouts and the task description without environment access, following mixture-of-agents aggregation (Lee et al., 2026b; Wang et al., 2025a). The *agentic verifier + revision* extends the agentic verifier to artifact editing: in a single context, it checks the candidates against the environment, chooses the best starting artifact, and corrects the errors it finds.

Selection and revision. The primary comparison selects an existing rollout, isolating verification judgment from generation and allowing comparison with the oracle. The full VERIHARNESS also applies the adjudicator’s revision plan to the selected base. This adds one editing pass without another investigation. Methods that generate a new artifact directly from the pool, such as aggregation agents (Lee et al., 2026b), are outside the selection comparison.

Metrics. Scores use each benchmark’s native scale: task success rate for APEX-Agents, mean rubric score for JobBench and Workspace-Bench Lite, the task-weighted mean across WorkBuddy Bench’s three domains, and headline accuracy for SpreadsheetBench 2. We report the three-seed mean, gain over the single-rollout score, and cross-seed standard deviation. The single-rollout mean and oracle are expectations over the fixed pool and require no resampling. Figure 6 in Appendix E plots the expected selection-oracle score as a function of the pool size N for both models.

E. Released Rollout Pool

We release the rollout pools used in this paper at <https://huggingface.co/datasets/caiqizh/veriharness> and the verification code at <https://github.com/google-research/veriharness>. The release covers both models on all five benchmarks with ten rollouts per task. For each rollout, it contains the trajectory, the delivered files, and the score assigned by the benchmark’s grader. The release contains no benchmark content: task descriptions, input workspaces, rubrics, and reference answers are not redistributed, and every rollout carries the official task identifier, so the inputs can be obtained from the original benchmark. Results that depend only on the pool can therefore be reproduced without running a model, including the single-rollout and selection-oracle rows of Table 2 and the curves of Figure 6.

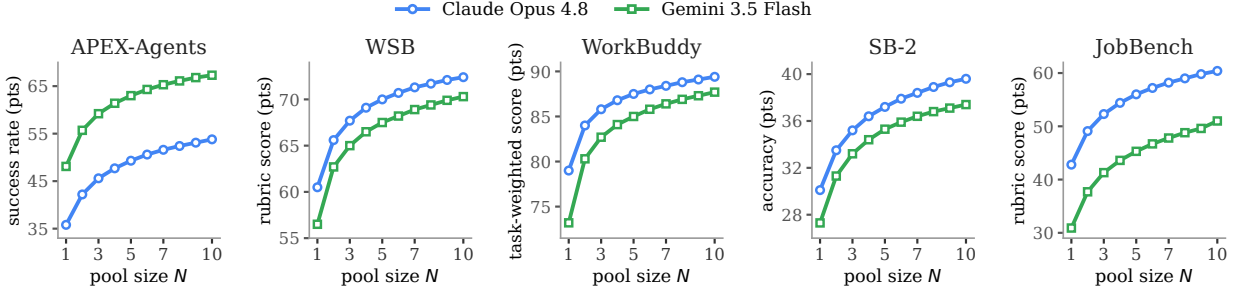


Figure 6 | **Selection oracle against pool size.** Expected score of the best rollout among N drawn without replacement from the pool of ten, on each benchmark’s own scale; $N=1$ is the single rollout and $N=10$ is the selection oracle of Table 2. The curves are computed from the grader scores in the released rollout pool.

F. Ablation of Harness Components

Table 4 runs the full harness with only one investigation active. In the resolver-only variant, adjudication receives $L_{\neq}$ alone and consensus claims are never challenged. In the challenger-only variant, it receives $L_{=}$ alone and disputed claims are left as the base rollout states them. Both variants use the same model, tools, skills, adjudication, and delivery as VERIHARNESS. The comparison applies the revision plan, because a consensus error is shared by every candidate and a challenger finding can therefore be realized only through revision. Two further variants keep both investigations: *without skills* runs the protocol with an empty skill library, and *single context* runs the resolver and challenger in one context, so that adjudication reads one combined record. Section 6 discusses the contributions of the resolver and the challenger. Removing the skill library lowers the score on every benchmark, with the largest loss on APEX-Agents. Merging the two investigations into one context costs less but lowers the score on every benchmark, which supports the separate contexts of Section 3.4.

Table 4 | **Score of the full harness with one part disabled.** Three-seed mean on each benchmark’s own scale after applying the revision plan; Avg is the unweighted mean over the five benchmarks.

Method	Avg	APEX	WSB	WorkBuddy	SB-2	JobBench
Gemini 3.5 Flash						
Single rollout	47.2	48.1	56.5	73.2	27.3	30.9
Resolver only	51.5	53.1	63.2	77.0	30.0	34.0
Challenger only	50.0	50.9	60.3	75.2	29.7	33.7
VERIHARNESS without skills	51.3	50.9	63.5	77.1	30.6	34.4
VERIHARNESS in a single context	52.6	53.9	64.5	77.9	31.3	35.3
VERIHARNESS	53.4	54.8	65.4	78.2	32.2	36.2
Claude Opus 4.8						
Single rollout	49.6	35.8	60.5	79.0	30.1	42.8
Resolver only	54.3	44.6	65.5	82.3	32.9	46.3
Challenger only	52.6	39.3	64.0	80.9	32.6	46.1
VERIHARNESS without skills	53.8	42.0	65.6	82.4	32.8	46.4
VERIHARNESS in a single context	55.3	46.1	66.5	83.3	33.5	47.0
VERIHARNESS	56.1	47.5	67.3	83.7	34.3	47.6

G. Why a Dedicated Runtime

VERIHARNESS’s verification protocol and skills can be deployed in either our dedicated runtime or an existing agent CLI. The CLI rows of Table 2 run the protocol inside existing agent CLIs. In the terms of Eq. (2), a CLI keeps its own generation harness $\mathcal{H}_G$ and receives the protocol Π and skills $\mathcal{S}$ as text in its instructions. These rows recover most of the gain, showing that the verification design transfers across runtimes. We nevertheless keep a dedicated runtime H_V for three reasons.

- *The program executes the protocol.* Running the two investigations in separate contexts, passing the evidence records to adjudication, and delivering the artifact with its verification record are steps that $\mathcal{H}_V$ executes explicitly. In our CLI deployments, protocol execution depends on the host agent’s operating loop, including its system instructions and context management. The weaker CLI results on JobBench and SpreadsheetBench 2 with Opus suggest that the host runtime can affect how effectively the verification protocol is carried out.
- *Explicit workspace and tool configuration.* $\mathcal{W}$ and $\mathcal{T}$ expose the N rollouts and the environment read-only, expose one writable output, render spreadsheets at cell level, and pass evidence records and revision plans between contexts as files. Our runtime configures these facilities explicitly and consistently across verification runs.
- *A controlled implementation.* Our runtime supports reproducibility and component ablations, as in Table 4, while the CLI results demonstrate the portability of the verification protocol and skills.

H. Tools and Skills

Table 5 | **Components of the verification harness.**

Part	What it provides	Examples
Workspace and tools ($\mathcal{W}, \mathcal{T}$)	Access to the objects under check	Read a file, recompute a table, trace a source
Verification protocol Π	Division of checking labor	Resolver, challenger, adjudication
Verification skills $\mathcal{S}$	Reusable knowledge of what to check	Period, unit, source version, required items

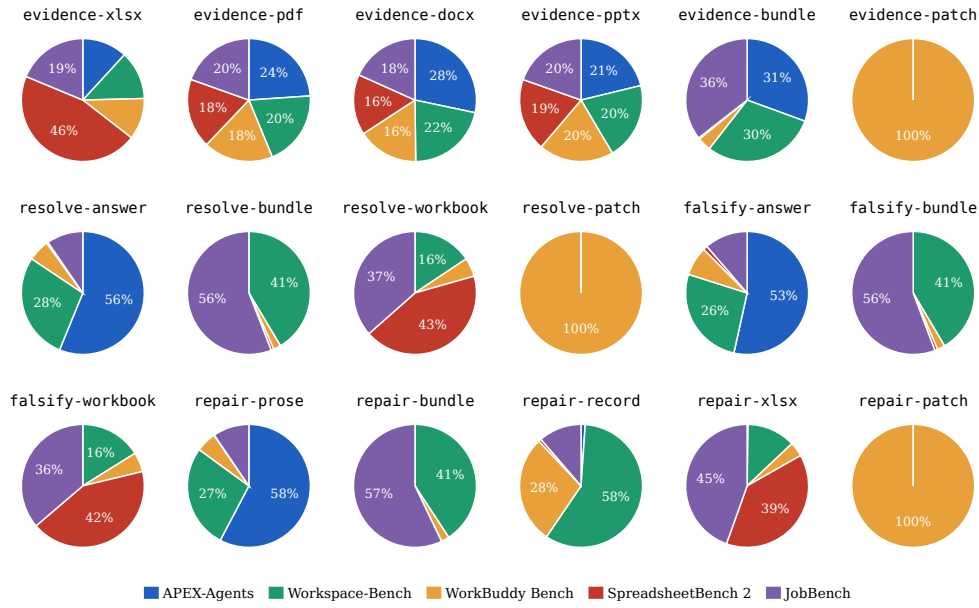


Figure 7 | **Where each skill was used.** For each of the eighteen skills, the number of invocations per task in the runs of Table 2 in our implementation, split by benchmark with both models pooled. We report rates rather than counts so that a large benchmark does not weigh more. An invocation is the skill being loaded into a context, its file being read by the model, or one of its scripts being run.

Tools. The verifier acts through the agent’s native tools: `read`, `ls`, `find`, and `grep` to look at the workspace and the rollouts, `bash` to run code, including the scripts a skill ships, and `write` and `edit`, which are restricted to the output directory, for its records and the files it delivers. This is what lets the same protocol and skills run unchanged inside Gemini CLI, Claude Code, and Codex (the CLI rows of Table 2).

Skills. Eighteen skills of four kinds are listed in Table 6. A skill is a short text file, optionally with scripts, written for one kind of deliverable. The six *evidence* skills tell the verifier how to read a workbook, PDF, document, deck, file bundle, or code patch as evidence rather than as extracted text, and ship the scripts for doing so. The four *resolver* skills say what counts as deciding evidence when candidates of a type disagree and in which order of authority to apply it. The three *challenger* skills list the ways a whole pool of that type goes wrong together, one check per entry. The five *revision* skills say how the type is graded, how an unresolved claim is closed, and what must never be edited.

Where the skills are used. Figure 7 shows, for each skill, how its use per task divides among the benchmarks. The document-format evidence skills are used at similar rates everywhere, except the spreadsheet skill, which SpreadsheetBench 2 draws on several times as often; the type-specific skills follow the deliverables each benchmark asks for, from written answers on APEX-Agents to patches and pages on WorkBuddy Bench.

Table 6 | **The skill library.** The eighteen skills of the human-authored library, the deliverable each is written for, what it supplies, and the scripts it ships.

Skill	Deliverable	What it supplies	Scripts
evidence-xlsx	any	formulas beside cached values, workbook diffs, rendered sheets	xlsx_dump, xlsx_diff, xlsx_gaps, xlsx_forks, xlsx_render
evidence-pdf	any	tables with their columns, words with coordinates, rendered pages	pdf_render, pdf_tables, pdf_words
evidence-docx	any	text in reading order, tables, tracked changes and comments	docx_changes, docx_text
evidence-pptx	any	shapes in slide order, tables, charts, notes, rendered slides	pptx_render, pptx_text
evidence-bundle	file bundle	inventory of what each candidate delivered and states	bundle_inventory
evidence-patch	patch or page	build and test of every candidate patch; headless-browser probe of a delivered page	patchlab, pageprobe
resolve-answer	written answer	which source, provision, and question governs a split	–
resolve-bundle	file bundle	the output contract, the basis of a count, enumerated lists, stored values	–
resolve-workbook	workbook	order of authority: input note cells, labels, sibling formulas, task words	–
resolve-patch	patch	executed behaviour, task words, repository idiom, project tests	–
falsify-answer	written answer	unopened provided documents, uncited governing clauses, statistics over the wrong set	–
falsify-bundle	file bundle	unused stated filenames, unsurfaced breakdowns, over-broad exclusions	–
falsify-workbook	workbook	blank labelled cells, filled unasked cells, scale, references, ranges, check rows	–
repair-prose	written answer	how prose is graded, what an unresolved claim permits, what never to change	–
repair-bundle	file bundle	how office bundles are graded, coverage against the other candidates	–
repair-record	structured record	how records are graded, filling unresolved fields, completing derived lists	–
repair-xlsx	workbook	what in-place edits break, recalculation, proof that edits touched only their targets	xlsx_recalc
repair-patch	patch	edit the tree, never the patch text; the patch must apply, reproduce, and pass what the base passed	–

I. Self-Evolution Details

Setup and split. The self-evolution experiments use Claude Opus 4.8 as both verifier and library proposer. All four conditions run the complete harness, including revision, with fixed rollout pools, protocol, and general tools; evolution changes only the skill library. The APEX-Agents experiment excludes file-deliverable tasks, and the SpreadsheetBench 2 experiment covers template and financial-model tasks. Each benchmark is partitioned once into about 75% development and 25% held-out tasks. All ten rollouts of a task stay in one partition, and tasks that share a source workspace, source documents, or a workbook template are grouped before splitting so that related instances cannot appear on both sides. The split uses a fixed seed, balances task categories as far as the grouping permits, and is shared by conditions A–D throughout. Figure 4 reports held-out scores under this setup, whereas Table 2 reports full-benchmark pools, so condition B need not match its full-harness row.

Information boundary. The human-authored library is written from general document-handling knowledge and development failures only, then frozen; its authors do not inspect held-out artifacts, traces, or grades. Human-authored and evolved skills contain reusable procedures rather than task identifiers or reference answers. Held-out grades are visible only to the evaluation and are never returned to the proposer or used to change prompts, skills, candidate counts, or stopping decisions. The verification run itself never sees reference answers or grading rubrics, so evolution uses development supervision while preserving the verification-time boundary of Section 4.

Learning loop. A development failure package contains the task and workspace, the fixed candidate rollouts, the harness’s evidence records and revision plan, the delivered artifact, and the grader’s per-item verdicts. From these packages the proposer generates K candidate libraries per round, with $K = 3$ at first and $K = 5$ later; candidates may include scripts, which run under a ten-minute limit. An automatic filter rejects candidates that contain task identifiers, benchmark names, development filenames, or reference-answer values. Each candidate and the current library are evaluated on the same development tasks and pools; the candidate with the largest nonnegative paired gain replaces the current library, and the library is kept when every candidate has a negative gain. The held-out score is recorded after this decision, so a retained update can lower it. The schedules of ten rounds for C and eight for D were fixed before any held-out trajectory was inspected, and the final libraries are those retained at the end of the schedule rather than checkpoints chosen by held-out score.

Round-by-round comparison. On APEX-Agents, C first reaches its final trajectory score of 49.6 at round 8, and D reaches that score at round 1. On SpreadsheetBench 2, C first reaches 38.5 at round 8, while D first exceeds it at round 7 with 39.4, so the reduction in rounds depends on the benchmark. Both trajectories include held-out regressions, consistent with selection on development scores, and candidate counts change across rounds, so round counts alone do not establish a compute or elapsed-time advantage. The figures report point estimates and do not establish statistical significance.

What the learned skills contain. The following procedures illustrate the learned libraries:

- **Requirement coverage.** For a written answer, enumerate required figures, list members, and elements of a governing rule. Compare the value supplied by each rollout for each item, then check the supporting evidence before accepting or revising the base artifact’s value.
- **Normalization.** For a normalized financial line, identify each named nonrecurring item and check that the adjustment reverses its contribution under the workbook’s conventions.
- **Period alignment.** For a growth rate, identify the start and end columns from the label and check that the formula uses that interval.
- **Sign conventions.** For a line labelled as a deduction, check how the workbook stores and applies

the sign before changing a value or formula.

Because evaluation scores the delivered artifact, the gains measure the combined effect of the learned skills on checking, adjudication, and delivery. The held-out comparisons show that the learned libraries help on unseen tasks within the evaluated splits; generalization beyond these task distributions, and improvement without development grader feedback, remain to be established.

J. Why Shared Errors Survive

This appendix supports the analysis of the consensus challenger in Section 6. On APEX-Agents with Claude Opus 4.8, we match each row of the challenger’s evidence record that concerns a claim shared by all ten rollouts to the rubric items it concerns, using the same model to read the record and the rubric. The archived per-item grades identify the claims on which every rollout was wrong and the challenger upheld the shared value. Table 7 classifies these surviving errors by the relation between the claim the challenger tested and the item the rubric grades.

Table 7 | **Why shared errors survive the challenger.** APEX-Agents, Claude Opus 4.8; share of the unanimous errors that the challenger upheld.

Relation between the tested claim and the graded item	Share
The tested claim is an input or intermediate quantity; the graded quantity lies downstream	70%
Same quantity, different definition, period, or source version	14%
The challenger confirmed the wrong value of the graded quantity	12%
Omission or mapping noise	4%

K. Compute Cost

This section compares the compute cost of the verification methods under a single accounting. Figure 8 plots each method’s compute per task against its gain over the single rollout for both models. VERIHARNES lies on the cost–gain frontier for both models, with and without revision. With Gemini 3.5 Flash, selection costs \$1.44 per task, a tenth of the cost of LLM-as-a-Verifier at twice the gain, and less than the pairwise tournament. With Claude Opus 4.8, selection costs \$3.92, about half the cost of LLM-as-a-Verifier, majority voting, and best-of- N , whose ten-rollout prompts are expensive at Opus prices, and a quarter of the cost of the tournament. Applying the revision plan costs about three times as much as selection and yields the largest gain in the figure. Running the same harness inside an agent CLI reaches the same gain with Flash at three times the cost of our runtime, and a lower gain with Opus at two to three times the cost. The harness is inexpensive because its calls are incremental turns over one workspace, so 86 to 91% of its input tokens are served from cache. When cached tokens are charged at the full input price, selection costs \$5.40 with Flash and \$13.06 with Opus, and the harness remains on the frontier, because the baselines re-read whole rollouts on every call and do not benefit from the cache.

Metering. Costs are computed from the token counts of the runs at list price, with cached input charged at the cache-read rate. Gains are the five-benchmark means of Table 2. LLM-as-a-Verifier is run with four evaluations per comparison. Majority voting, best-of- N , and the CLI runs are estimates from their call counts and context sizes rather than metered runs and are shown with hollow markers and an asterisk; the aggregation baseline calls its own client and could not be metered.

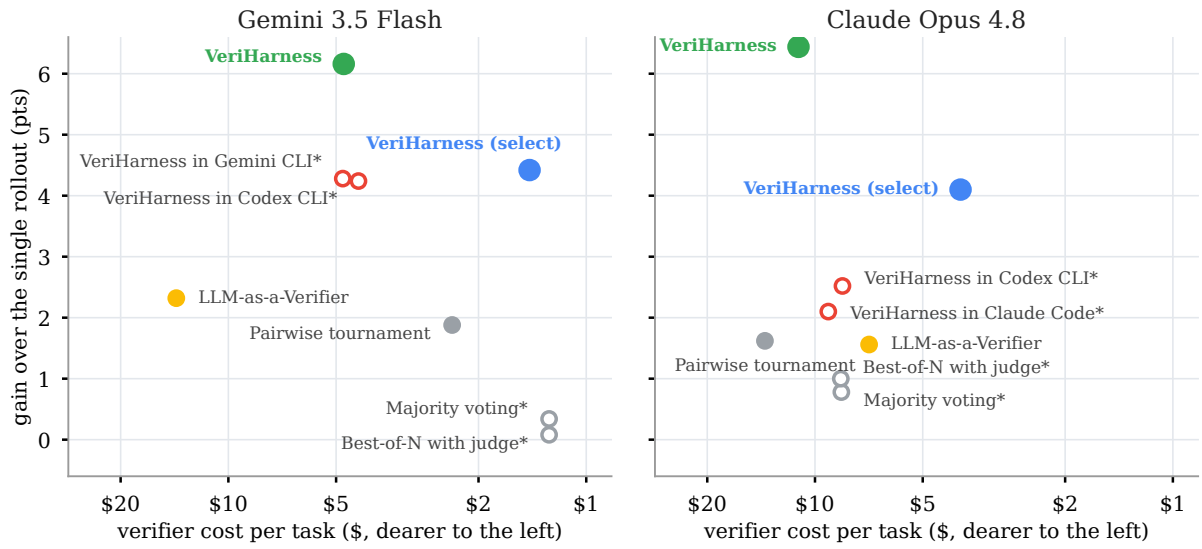


Figure 8 | **Cost against gain.** Compute per task at list price with cached input at the cache-read rate, on a log scale with cost decreasing to the right, against the gain over the single rollout, the five-benchmark mean of Table 2. Hollow markers with an asterisk are estimates.